

Product Reference Manual (V1.0)

Description

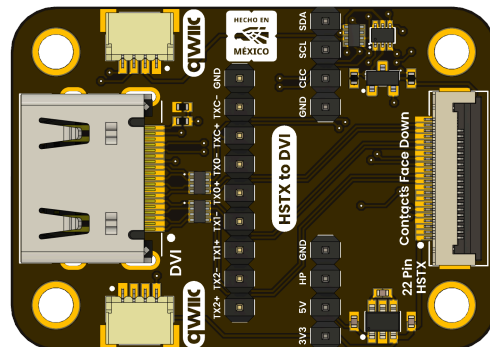
The DevLab DVI to FPC Adapter is a passive interface board designed to support embedded video generation workflows, enabling direct access to DVI signals for development, validation, and signal analysis. It exposes all connector pins while maintaining the electrical characteristics required for high-speed TMDS transmission, making it suitable for software-defined video implementations on microcontrollers.

The board integrates a 22-pin 0.5 mm FPC connector compatible with HSTX-based interfaces, allowing direct coupling with platforms such as RP2040/RP2350 running DVI output stacks (e.g., PicoDVI). This enables real-time TMDS-encoded video transmission driven by PIO, DMA, or dedicated hardware blocks. Additionally, the DDC channel is routed to a QWIIC (JST 1 mm) connector, providing standardized I²C access for EDID retrieval and DDC/CI communication.

As a fully passive pass-through design, the adapter performs no signal conditioning or protocol translation. It preserves impedance and signal integrity across TMDS differential pairs while exposing auxiliary lines such as CEC, HPD, and I²C, allowing low-level interaction with both the video stream and display control layers.

DevLab format compatibility.

Simplicity and compatibility are the core principles of the **DevLab form factor**. This standard defines a **compact and communication-optimized board layout**, ensuring straightforward connection and interoperability among DevLab modules.



Key Features

- Passive DVI signal breakout with full pin exposure
- 22-pin 0.5 mm FPC connector compatible with HSTX DVI implementations
- QWIIC (JST 1 mm) connector for DDC / I²C access
- Supports EDID reading and DDC/CI communication
- Dual connection paths: FPC flex cable or 2.54 mm pin headers
- Preserves signal integrity for high-speed TMDS differential pairs
- Compatible with QWIIC, DevLab, and PiicoDev ecosystems

Hardware Features

- **DVI Connector**
 - Breaks out all 19 standard DVI signals
 - Maintains controlled routing for TMDS differential pairs
- **FPC Interface**
 - 22-pin, 0.5 mm pitch FPC connector
 - Optimized for HSTX-based DVI signal routing
- **Pin Header Access**
 - 2.54 mm pitch through-hole pads for jumper wires, probing, and logic analysis
- **QWIIC Connector**
 - JST 1 mm, 4-pin interface
 - Routes DDC signals (SCL, SDA) and power rails
- **Control Signal Access**
 - CEC, HPD, and DDC lines exposed for monitoring and control

Applications

- Embedded video output for microcontroller-based systems
- Heads-Up Displays (HUDs) and simple UI rendering
- Custom display controllers and diagnostics tools
- EDID reading and monitor feature analysis
- DVI cable testing and signal integrity verification
- Home automation via DVI-CEC control

Software Support

The adapter does not require firmware or drivers; however, it is designed to support advanced software implementations on compatible microcontrollers:

- **RP2040 / RP2350**
 - HSTX and PIO-based DVI signal generation
 - Low-resolution video output, test patterns, and simple graphics
 - Direct EDID and DDC/CI communication via QWIIC
- **I²C / DDC Tools**
 - EDID parsing and monitor capability identification
 - DDC/CI command support for display configuration

CONTENTS

Description	1
DevLab format compatibility.	1
Key Features	1
Hardware Features	2
Software Support	2
Applications	2
1 The Board	4
1.1 Accessories	4
2 Ratings	4
2.1 Recommended Operating Conditions	4
Board Functional Sections	6
3.2 Board Topology	9
4 Connectors & Pinouts	10
4.1 General Pinout	10
4.2 Pinout General Description	11
Notes	12
5 Board Operations	13
TMDS Signal to GPIO Mapping	13
Physical Connection (FPC / DevLab DVI)	13
Firmware Configuration	14
Connection Reference (Raspberry Pi Pico Header)	14
Design Considerations	14
Compatibility Notes	14
Expected Behavior	14
PicoDVI Example	15
1. Install the PicoDVI Library	15
2. Prepare the Code	16
4. Upload the Firmware	19
5. Verify Operation	19
Expected Result	20
6 Mechanical Information	21
7 Company Information	22
8 Reference Documentation	22

1 The Board

1.1 Accessories

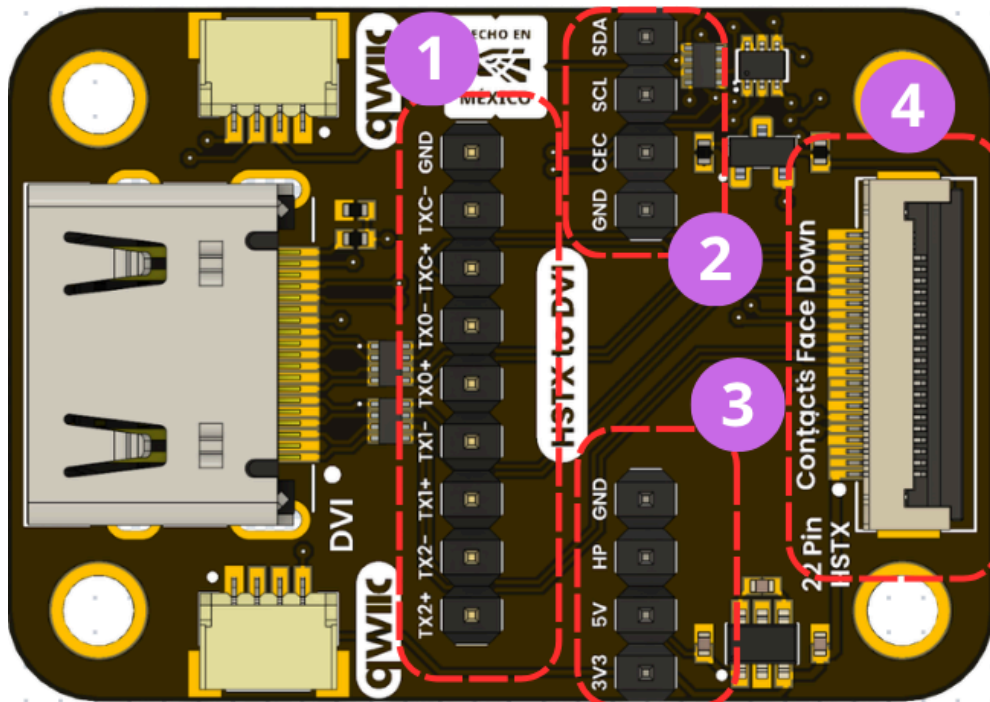
2 Ratings

2.1 Recommended Operating Conditions

Symbol	Description	Min	Typ	Max	Unit
VDD	Operating supply voltage	—	3.3	5.5	V
VCC	Input supply voltage (external input via VCC pin)	3.6	5.0	6.0	V
VIL	Low-level input voltage (I ² C interface)	-0.3	—	0.99	V
VIH	High-level input voltage (I ² C interface)	2.31	—	5.2	V
VOL	Low-level output voltage (at IOL = 3 mA)	—	—	0.4	V
VOH	High-level output voltage (at IOH = -3 mA)	2.4	—	3.3	V

3 Functional Overview

The DevLab DVI to FPC Adapter enables simultaneous DVI video output and DDC (I²C) communication between a microcontroller and a DVI/HDMI display through a fully passive signal path. When paired with capable devices such as RP2040 or RP2350, the system supports real-time video generation while concurrently accessing display metadata via EDID.



Concurrent Video Output and DDC Operation

The adapter supports parallel operation of high-speed video and low-speed control channels:

- TMDs differential pairs are routed directly from the microcontroller to the DVI connector, enabling continuous video transmission
- DDC signals (SCL/SDA) are routed simultaneously to both breakout headers and the QWIIC connector

This architecture allows EDID communication during active video transmission, without requiring multiplexing or hardware-level arbitration.

EDID Detection and Monitor Capability Discovery

Through the exposed DDC interface, a microcontroller can automatically detect a connected monitor at the standard **EDID address (0x50)** and retrieve complete display metadata using the DDC2B protocol. Typical operations include:

- Automatic monitor presence detection
- Full EDID block acquisition (base + extension blocks)

- Validation of EDID checksum
- Parsing of manufacturer ID, product code, serial number, and manufacture date
- Extraction of supported resolutions, refresh rates, and timing descriptors
- Identification of extension blocks such as CEA-861

This information can be used to adapt rendering modes, validate compatibility, or display diagnostic information directly on the monitor.

Microcontroller-Based DVI Rendering

When used with microcontrollers supporting high-speed GPIO or video-capable peripherals (e.g. PIO or HSTX), the adapter enables native DVI signal generation.

Validated implementations include:

- Real-time framebuffer rendering
- Resolution scaling (e.g. 320×240 → 640×480 @ 60 Hz)
- Embedded-friendly color formats (e.g. RGB565)
- Basic graphical primitives (text, shapes, test patterns)

All video timing, encoding, and rendering are handled entirely by the host microcontroller. The adapter introduces no processing, buffering, or scaling.

Board Functional Sections

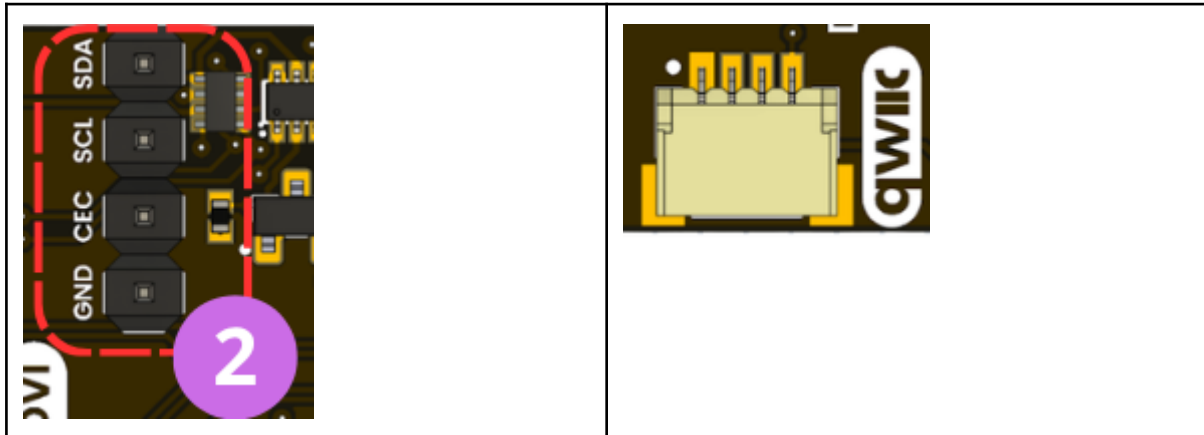
DVI Breakout & TMDS Channels (PIO Mapping)



Direct access to all DVI signals, including TMDS differential pairs (TX0, TX1, TX2, TXC). These lanes are intended to be driven by MCU GPIOs (RP2040 / RP2350) using PIO or HSTX for real-time TMDS generation.

Consistent lane color coding is recommended for debugging and signal tracing.

Control & DDC Interface (I²C / QWIIIC)

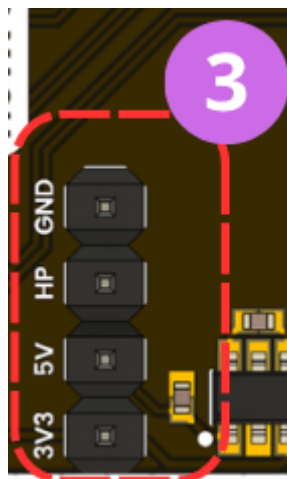


Includes all low-speed control signals:

- DDC (SCL / SDA)
- CEC
- HPD

DDC is available on both headers and QWIIIC, enabling EDID access and DDC/CI communication. HPD provides connection status, while CEC supports single-wire control.

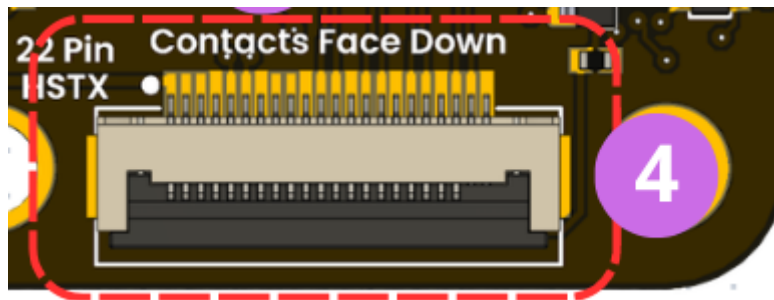
Power & Reference Rails



Provides access to:

- 3.3V
- 5V
- GND

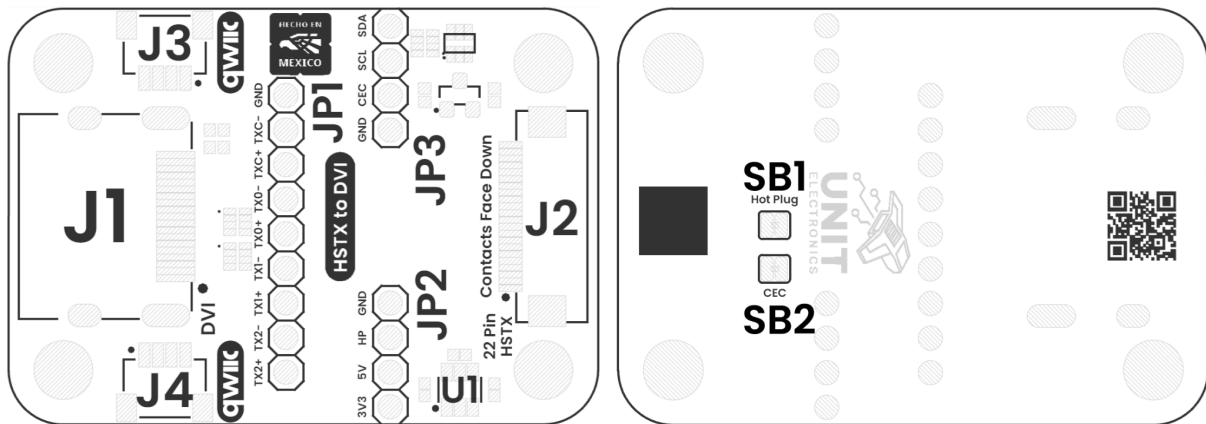
Used for electrical validation, interfacing, and measurement during development.

HSTX FPC Interface (High-Speed Video Interface)

The 22-pin 0.5 mm FPC connector is designed for HSTX-compatible interfaces, enabling direct connection to microcontrollers such as RP2350. This interface supports high-speed digital video transmission driven entirely by the host system.

It allows compact integration and direct routing of video data paths, making it suitable for software-defined DVI implementations without requiring an external PHY.

3.2 Board Topology



Top Side

Bottom Side

Views of Board Topology

Views of DRV2605L Haptic Motor Controller Module Topology

Table 3.2.1 - Components Overview

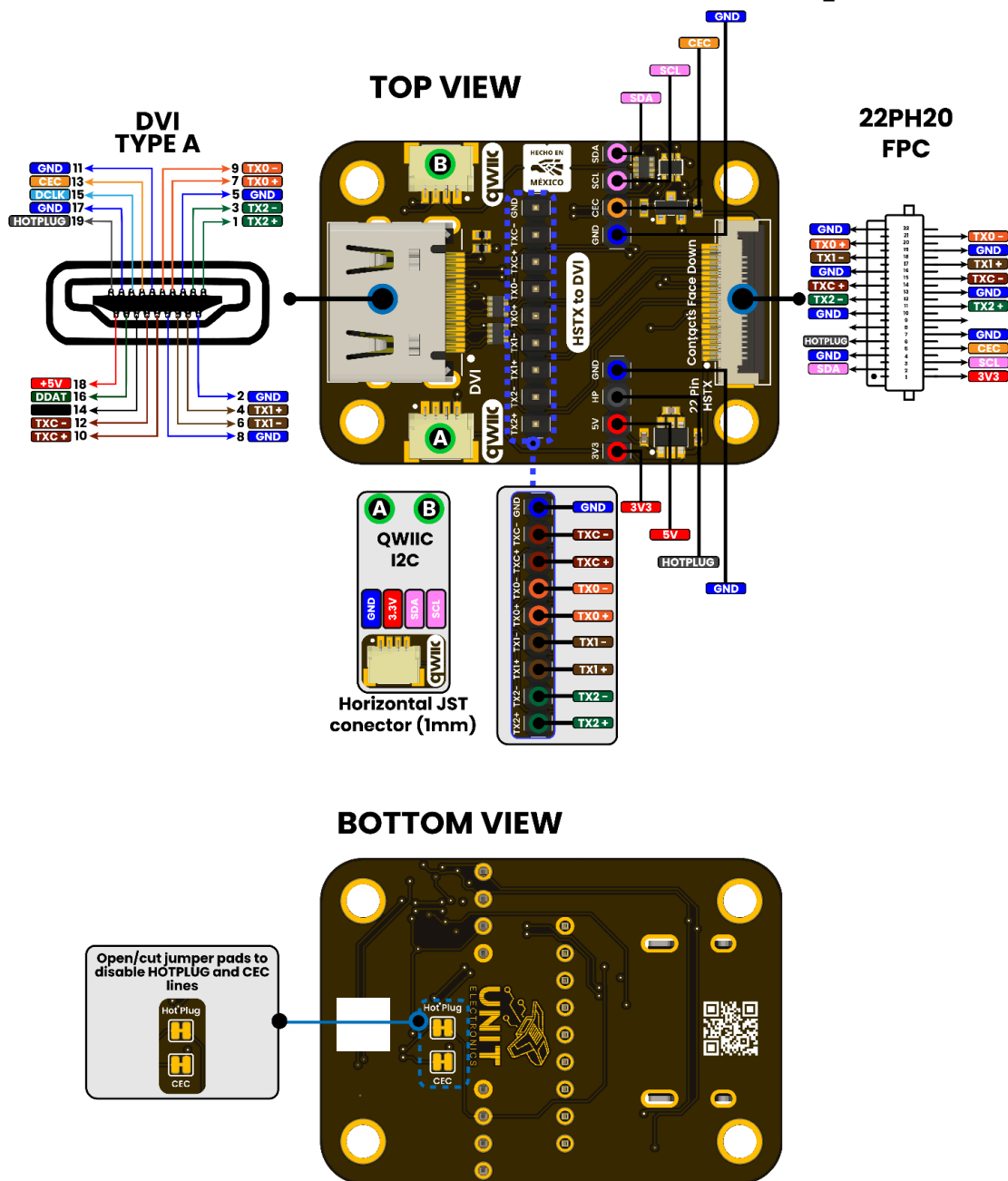
Ref.	Description
J1	DVI Connector
J2	FPC 22-pin 0.5mm Connector
J3	I2C QWIIC Connector (JST 4-Pin 1mm)
J4	I2C QWIIC Connector (JST 4-Pin 1mm)
U1	AP3602A 5V Regulator
JP1	2.54 mm Pin Header for TMDS Signals
JP2	2.54 mm Pin Header for Power Supply and Hotplug
JP3	2.54 mm Pin Header for I2C and CEC Signals
SB1	Jumper pad for Hotplug
SB2	Jumper pad for CEC

4 Connectors & Pinouts

4.1 General Pinout

PINOUT

DevLab: DVI to FPC Adapter



DevLab Module General Pinout

4.2 Pinout General Description

Pin Label	Function	Notes
+3V3	Power Supply	3.3V power supply for logic and auxiliary circuits
GND	Ground	Common ground reference
SCL	I2C SCL	Serial Clock Line (DDC / EDID communication)
SDA	I2C SDA	Serial Data Line (DDC / EDID communication)
TX0+	TMDS Data 0+	Differential data lane 0 (positive)
TX0-	TMDS Data 0-	Differential data lane 0 (negative)
TX1+	TMDS Data 1+	Differential data lane 1 (positive)
TX1-	TMDS Data 1-	Differential data lane 1 (negative)
TX2+	TMDS Data 2+	Differential data lane 2 (positive)
TX2-	TMDS Data 2-	Differential data lane 2 (negative)
TXC+	TMDS Clock+	Differential clock lane (positive)
TXC-	TMDS Clock-	Differential clock lane (negative)
CEC	HDMI CEC	Consumer Electronics Control (optional, not used in basic DVI mode)
HOTPLUG	Hot Plug Detect	Indicates display connection status
5V	Power Input	5V supply from HDMI/DVI connector (used for detection / EDID)
HPD	Hot Plug Detect	Alternate label for HOTPLUG (depending on schematic naming)
NC	Not Connected	Reserved / not used

Notes

- TMDS signals must be routed as controlled impedance differential pairs
- I2C (SCL/SDA) is used for EDID reading (DDC channel)
- **HOTPLUG** is required for proper display detection in some monitors
- **CEC** is optional and typically unused in PicoDVI implementations
- **5V** is input from display, not for powering the board

5 Board Operations

5.1 Getting Started

This section describes the direct connection between the DVI TMDS signals exposed on the FPC connector and the GPIOs of the RP2040. This mapping allows users to generate DVI video using the PicoDVI library via the PIO peripheral.

TMDS Signal to GPIO Mapping

TMDS Signal	Differential Pair	RP2040 GPIO	Direction	Description
TX0+	Data Channel 0	GPIO8	Output	TMDS Data Lane 0 (positive)
TX0-	Data Channel 0	GPIO9*	Output	TMDS Data Lane 0 (negative)*
TX1+	Data Channel 1	GPIO12	Output	TMDS Data Lane 1 (positive)
TX1-	Data Channel 1	GPIO13*	Output	TMDS Data Lane 1 (negative)*
TX2+	Data Channel 2	GPIO14	Output	TMDS Data Lane 2 (positive)
TX2-	Data Channel 2	GPIO15*	Output	TMDS Data Lane 2 (negative)*
TXC+	Clock Channel	GPIO10	Output	TMDS Clock (positive)
TXC-	Clock Channel	GPIO11*	Output	TMDS Clock (negative)*
GND	—	GND	—	Signal reference

***Note:** The negative signals (TXx-) are internally generated by the PIO serializer using differential encoding. In most PicoDVI configurations, only the primary GPIOs listed in the firmware (`pins_tmds` and `pins_clk`) are required. The complementary signals are handled by the hardware mapping and resistor network.

Physical Connection (FPC / DevLab DVI)

The FPC connector exposes all TMDS lanes directly. When using the DualMCU v2 board:

- The routing between GPIO and TMDS pairs is already handled on the PCB
- No external level shifting is required
- Output is **3.3V logic adapted to TMDS via resistor network**

Firmware Configuration

The following configuration must match the GPIO mapping:

```
static const struct dvi_serialiser_cfg devlab_dvi_cfg = {
    .pio = pio0,
    .sm_tmds = {0, 1, 2},
    .pins_tmds = {8, 12, 14},
    .pins_clk = 10,
    .invert_diffpairs = false
};
```

Connection Reference (Raspberry Pi Pico Header)

Pico Pin	GPIO	Function
Pin 11	GPIO8	TMDS Data 0
Pin 16	GPIO12	TMDS Data 1
Pin 19	GPIO14	TMDS Data 2
Pin 14	GPIO10	TMDS Clock
Any GND	—	Ground

Design Considerations

- GPIOs must match the PIO timing requirements for TMDS generation
- Keep traces short and impedance consistent for signal integrity
- Avoid reassigning these GPIOs to other peripherals during operation
- Ensure stable 3.3V supply during video transmission
- Overclocking may be required depending on resolution

Compatibility Notes

- Fully compatible with **RP2040 (Raspberry Pi Pico)**
- Compatible with **RP2350 (with HSTX support improvements)**
- Works with PicoDVI-based libraries (Adafruit / uPicoDVI)

Expected Behavior

When correctly connected and configured:

- The display initializes automatically
- A stable DVI signal is generated

- The monitor locks at **320×240 @ 60 Hz**

PicoDVI Example

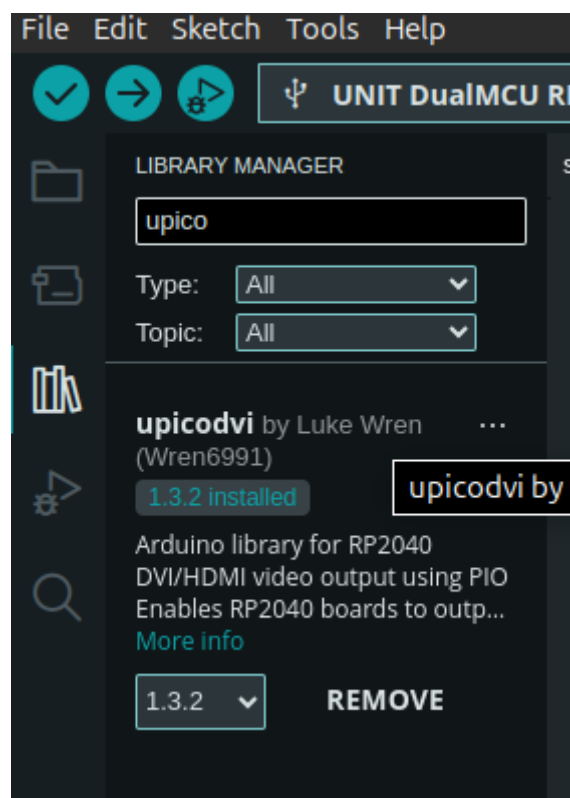
This section describes how to run the DVI test using the Arduino IDE and the PicoDVI library.

1. Install the PicoDVI Library

1. Open **Arduino IDE**
2. Go to **Sketch** → **Include Library** → **Manage Libraries...**
3. In the search bar, type:

`upicodvi`

4. Install the **uPicoDVI** library (make sure version is compatible with RP2040)



2. Prepare the Code

1. Open a new Arduino sketch
2. Copy and paste the provided test code (PicoDVI animation example)

```
// Double-buffered 8-bit Adafruit_GFX-compatible framebuffer for
PicoDVI

// Flicker-free animation with palette-based rendering

#include <upicodvi.h>

static const struct dvi_serialiser_cfg devlab_dvi_cfg = {

    .pio = pio0,

    .sm_tmds = {0, 1, 2},

    .pins_tmds = {8, 12, 14},

    .pins_clk = 10,

    .invert_diffpairs = false
};

DVIGFX8 display(DVI_RES_320x240p60, true, devlab_dvi_cfg);

#define N_BALLS 100

#define BALL_RADIUS 20

struct Ball {

    int16_t x;

    int16_t y;

    int8_t vx;

    int8_t vy;

};

Ball ball[N_BALLS];
```



```
void setup() {

  if (!display.begin()) {

    pinMode(LED_BUILTIN, OUTPUT);

    for (;;) {

      digitalWrite(LED_BUILTIN, (millis() / 500) & 1);

    }

  }

  randomSeed(millis());

  for (int i = 0; i < N_BALLS; i++) {

    display.setColor(i + 1,

                     64 + random(192),

                     64 + random(192),

                     64 + random(192));

    ball[i].x = BALL_RADIUS + random(display.width() - 2 * BALL_RADIUS);

    ball[i].y = BALL_RADIUS + random(display.height() - 2 *
BALL_RADIUS);

    do {

      ball[i].vx = 2 - random(5); // range: -2 to +2

      ball[i].vy = 2 - random(5);

    } while ((ball[i].vx == 0) && (ball[i].vy == 0));

  }

  display.setColor(255, 255, 255, 255); // palette entry 255 = white

  display.swap(false, true);
```

```

}

void loop() {

    display.fillScreen(0);

    for (int i = 0; i < N_BALLS; i++) {

        display.fillCircle(ball[i].x, ball[i].y, BALL_RADIUS, i + 1);

        ball[i].x += ball[i].vx;

        ball[i].y += ball[i].vy;

        if (ball[i].x <= BALL_RADIUS) {

            ball[i].x = BALL_RADIUS;

            ball[i].vx *= -1;

        } else if (ball[i].x >= (display.width() - 1 - BALL_RADIUS)) {

            ball[i].x = display.width() - 1 - BALL_RADIUS;

            ball[i].vx *= -1;

        }

        if (ball[i].y <= BALL_RADIUS) {

            ball[i].y = BALL_RADIUS;

            ball[i].vy *= -1;

        } else if (ball[i].y >= (display.height() - 1 - BALL_RADIUS)) {

            ball[i].y = display.height() - 1 - BALL_RADIUS;

```

```

    ball[i].vy *= -1;

}

}

display.swap();
}

```

3. Verify that the GPIO configuration matches your hardware:

```

.pins_tmds = {8, 12, 14},
.pins_clk = 10,

```

3. Select the Board

- ☐ Go to **Tools** → **Board**
- ☐ Select:
 - Raspberry Pi Pico / RP2040
- ☐ Select the correct **port**

4. Upload the Firmware

1. Connect your board via USB
2. Click **Upload**
3. Wait until the compilation and flashing process completes

5. Verify Operation

After uploading:

- Connect the DVI output to a monitor
- The display should show a **moving animation**
- Resolution: **320×240 @ 60 Hz**

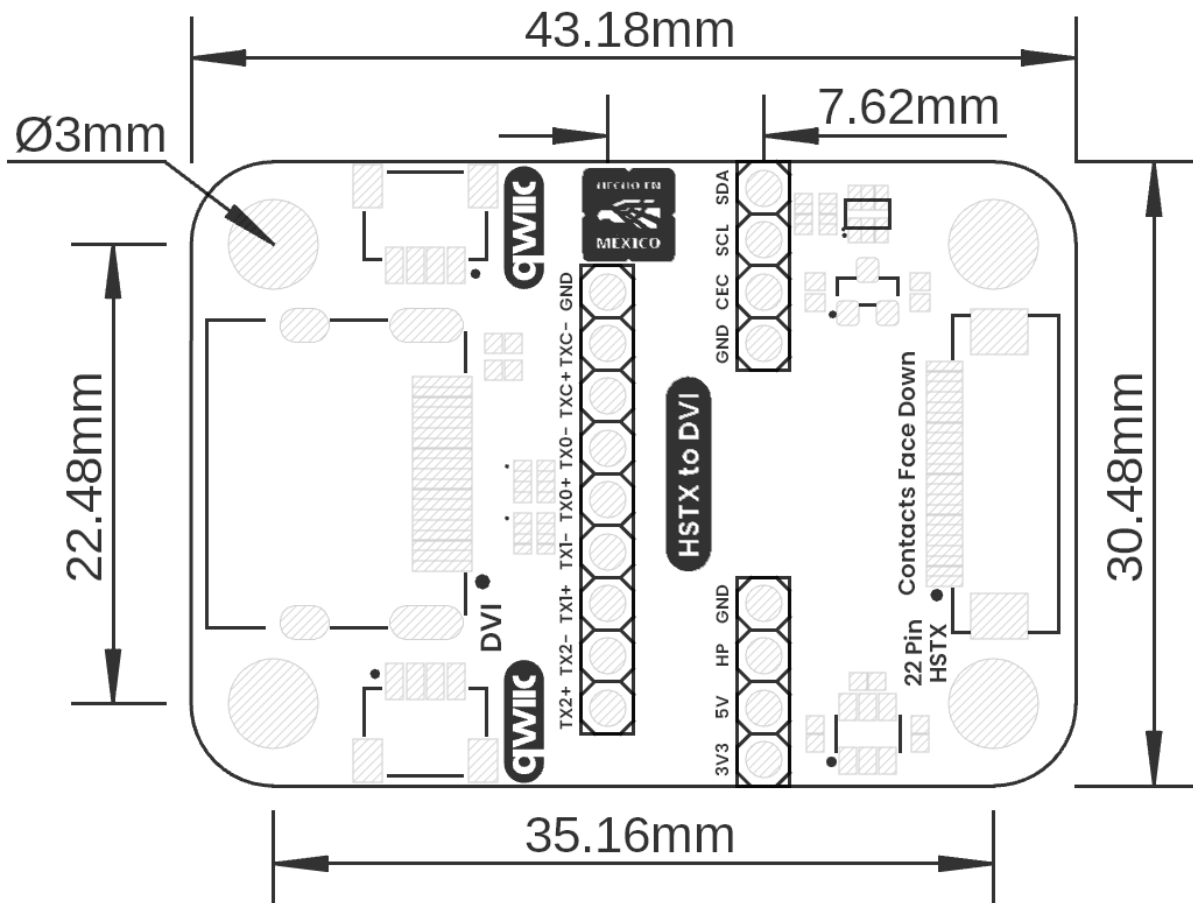


Expected Result

- Stable video signal
- No flickering or artifacts
- Continuous animation on screen



6 Mechanical Information



Board Dimensions in Millimeters

Mechanical dimensions in millimeters

7 Company Information

Company name	UNIT Electronics
Company website	https://uelectronics.com/
Company Address	Salvador 19, Cuauhtémoc, 06000 Mexico City, CDMX

8 Reference Documentation

Ref	Link
Documentation	https://github.com/UNIT-Electronics-MX/devlab_dvi_to_fpc_adapter
Getting Started Guide	https://wiki.uelectronics.com/wiki/devlab-dvi-to-fpc-adapter
Github Library	https://github.com/UNIT-Electronics-MX/unit_picodvi_library
Thonny IDE	https://thonny.org/
Arduino IDE	https://www.arduino.cc/en/software
Visual Studio Code	https://code.visualstudio.com/download

9 Appendix

9.1 Schematic (https://github.com/UNIT-Electronics-MX/unit_devlab_dvi_to_fpc_adapter/blob/main/hardware/unit_sch_v_1_0_0_ue0117_devlab_dvi_to_fpc_adapter.pdf)

